

UNIT-II Computer Arithmetic

Numbers can be represented in 3 different ways.

1) signed integer

• 3 major presentation

1) Sign and magnitude

2) one's complement

3) 2's complement

Addition and subtraction 2's complement:

$$\begin{array}{r} 4 \\ -13 \\ \hline 4 \end{array} \quad \begin{array}{r} 0100 \\ 0011 \\ \hline 0111 \end{array} \quad \begin{array}{r} -4 \\ +(-3) \\ \hline -4 \end{array} \quad \begin{array}{r} 1100 \\ 1101 \\ \hline 11001 \end{array}$$

$$\begin{array}{r} 4 \\ -3 \\ \hline 1 \end{array} \quad \begin{array}{r} 0100 \\ 1101 \\ \hline 10001 \end{array} \quad \begin{array}{r} -4 \\ +3 \\ \hline -1 \end{array} \quad \begin{array}{r} 1100 \\ 0011 \\ \hline 1111 \end{array}$$

→ simpler addition schema makes two complement

the most common choice for integer number systems within digital systems.

→ By using the concepts of complements, Addition becomes subtraction so there is no use of separate hardware for subtraction operation.

→ 2's complement representation is most widely used.

Addition, subtraction of signed magnitude data.

operation	Add magnitude	subtract magnitude		
		A > B	A < B	A = B
(+A) + (+B)	+(A+B)			
(+A) + (-B)		+(A-B)	-(B-A)	+(A-B)
(-A) + (+B)		-(A-B)	+(B-A)	+(A-B)
(-A) + (-B)	-(A+B)			
(+A) - (+B)		+(A-B)	-(B-A)	+(A-B)
(+A) - (-B)	+(A+B)			
(-A) - (+B)	-(A+B)			
(-A) - (-B)		-(A-B)	+(B-A)	+(A-B)

UNIT-II Computer Arithmetic

Numbers can be represented in 3 different ways.

1) signed integer.

• 3 major presentation.

1) Sign and magnitude

2) one's complement

3) 2's complement

Addition and subtraction 2's complement:

$$\begin{array}{r} 4 \\ + 3 \\ \hline 7 \end{array}$$

$$\begin{array}{r} 0100 \\ 0011 \\ \hline 0111 \end{array}$$

$$\begin{array}{r} -4 \\ + (-3) \\ \hline -7 \end{array}$$

$$\begin{array}{r} 1100 \\ 1101 \\ \hline 11001 \end{array}$$

$$\begin{array}{r} 4 \\ - 3 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 0100 \\ 1101 \\ \hline 10001 \end{array}$$

$$\begin{array}{r} -4 \\ + 3 \\ \hline -1 \end{array}$$

$$\begin{array}{r} 1100 \\ 0011 \\ \hline 1111 \end{array}$$

→ simpler addition schema makes two complement the most common choice for integer number systems within digital systems.

→ By using the concepts of complements, Addition becomes subtraction so there is no use of separate hardware for subtraction operation.

→ 2's complement representation is most widely used.

Addition, subtraction of signed magnitude data.

operation	Add magnitudes	subtract magnitude		
		A > B	A < B	A = B
(+A) + (+B)	+(A+B)	+(A-B)	-(B-A)	+(A-B)
(+A) + (-B)		-(A-B)	+(B-A)	+(A-B)
(-A) + (+B)				
(-A) + (-B)	-(A+B)	+(A-B)	-(B-A)	+(A-B)
(+A) - (+B)				
(+A) - (-B)	+(A+B)			
(-A) - (+B)	-(A+B)			
(-A) - (-B)		-(A-B)	+(B-A)	+(A-B)

Addition Algorithm:

- when the sign's of A and B are identical, add the two magnitudes and give sign of A to the result.
- When the sign's of A and B are different, compare the magnitudes and subtract smaller number from the larger number, the sign of the result is same as A if $A > B$, or the complement, the sign of A' if $A < B$.
- If Both magnitude are equal subtract B' from A and the result is positive.

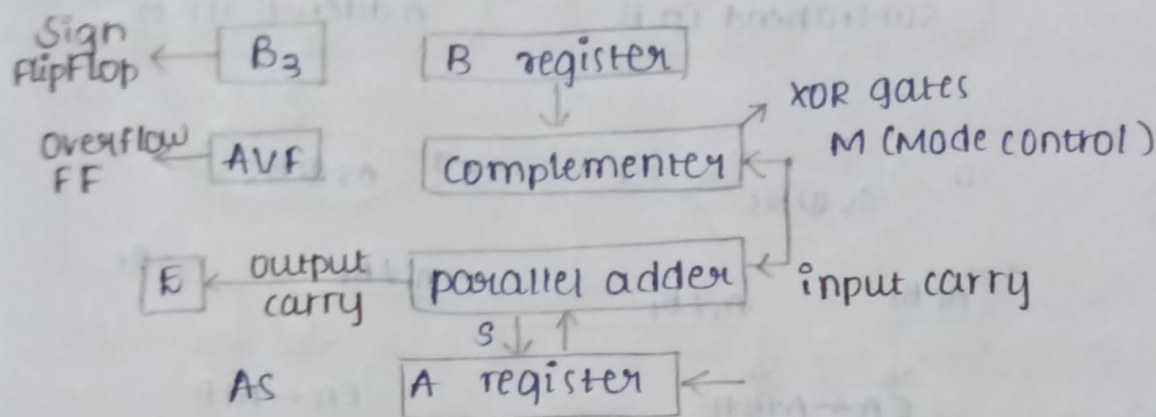
Subtraction Algorithm:

- when the signs of A and B are different, Add the two magnitudes and give the sign of A' to the result.
- When the sign's of A and B are identical, compare the magnitudes and subtract the smaller number from the larger number and the sign of the result is same as A if $A > B$, and if $A < B$ complement of A' and if $A = B$, subtract B from A and the result is '+ve'.

Hardware implementation for sign magnitude Addition and subtraction.

- Let A and B are the two registers that holds the magnitudes of the two numbers and $A's$ and $B's$ are the two flipflops used to store the corresponding signs.
- The output is stored in $A's$ and A which are combinedly called as Accumulator.
- As per the algorithm/table, we require an Adder, magnitude comparator and two subtractors.
- But Alternatively, the subtraction may be implemented by means of complement of Addition.

→ Hence, we require only a parallel adder and the complementer.



→ The complementer provides either B or B' depending on the mode controller - 'M'

if $M=0$ $A+B+0 \Rightarrow$ Addition

$M=1$ $A+B'+1 \Rightarrow$ subtraction

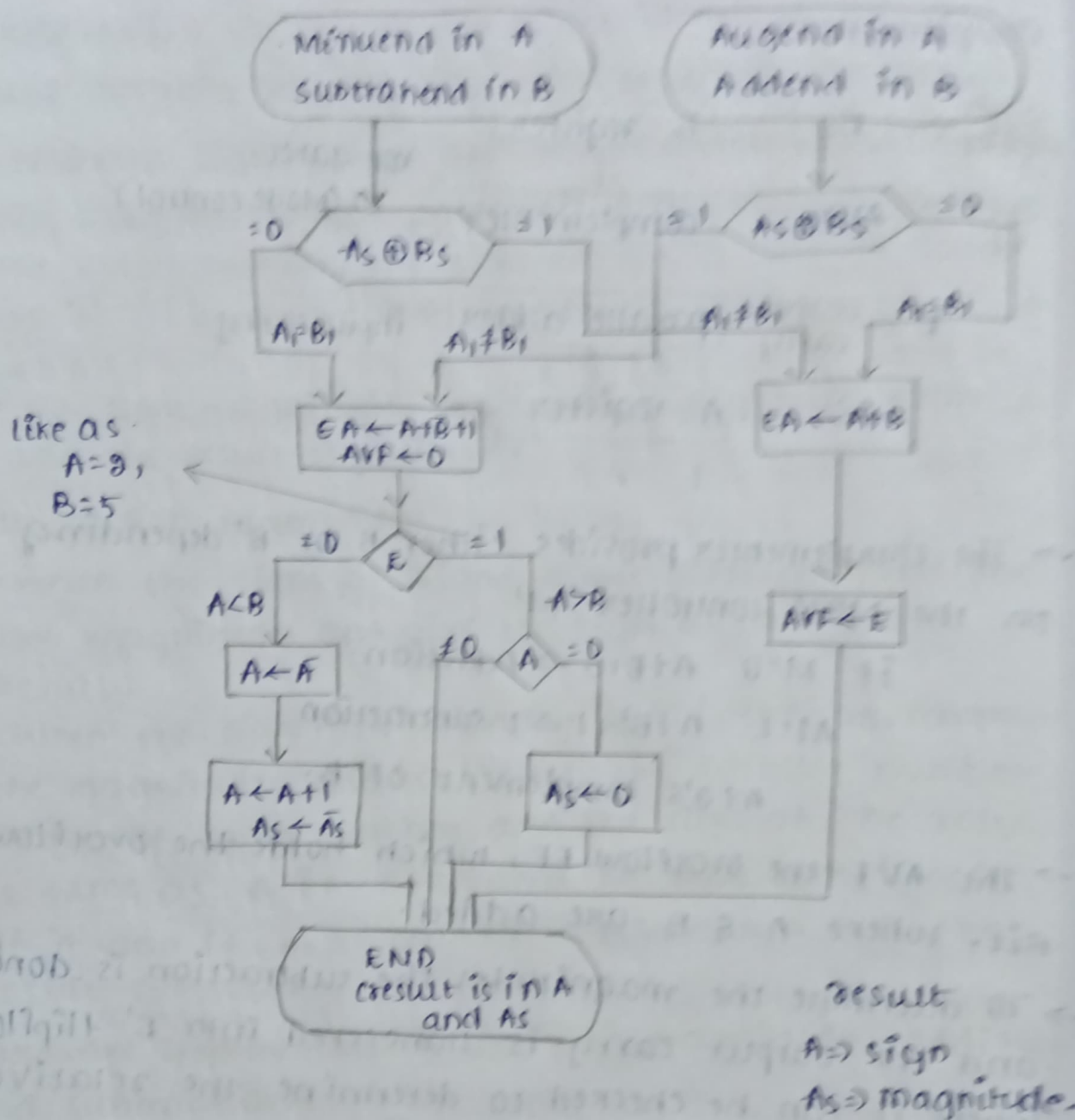
A + A 's complement of B .

→ The AVF the overflow FF, which holds the overflow Bit, where A & B are added.

→ To determine the magnitudes the subtraction is done and the output carry is transferred into 'E' flipflop where it can be checked to determine the relative magnitudes of two numbers.

Subtraction operation

Add operation

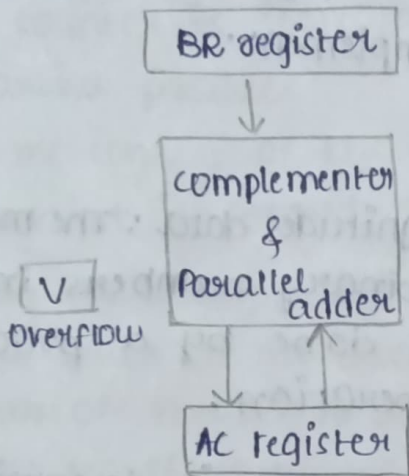


Addition and subtraction with signed 1's complement data:

- The addition of two numbers in signed 1's complement form consists of adding the number with the sign bits treated the same as the other bits of the number
- A carry out of the sign bit position is discarded.
- The subtraction consists of first taking the 1's complement of the subtrahend and then adding it to the minuend.

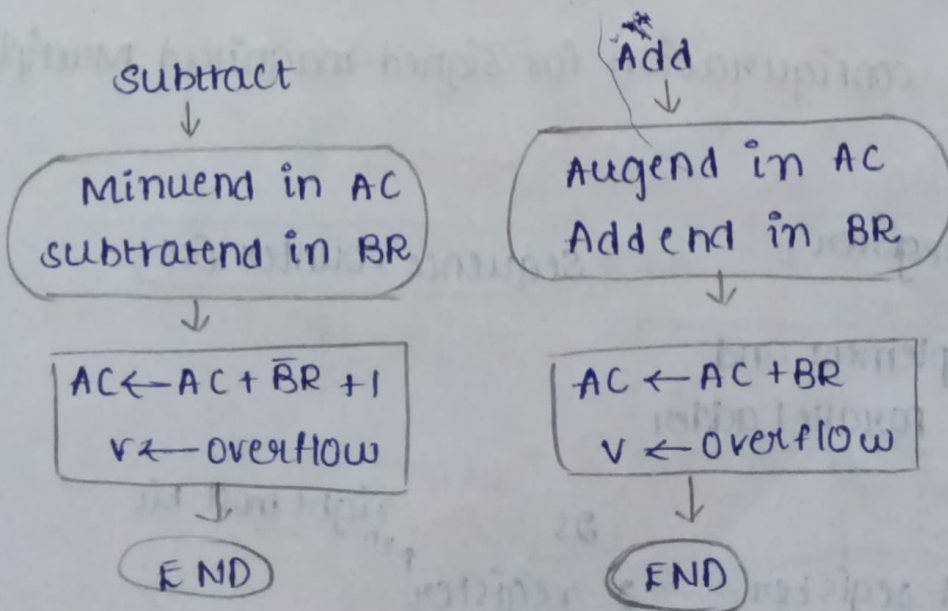
Hardware Configuration

Hence, there is no need to separate the sign bits, so a single register is sufficient to store both sign and magnitude.



(Fig) Hardware for sign's 2's complement addition and subtraction.

- 1) - The left most bit in AC and BR represents the sign bits of the numbers.
- 2) - The overflow flip-flops V is set to 1 there is an overflow. The output carry in this case is discarded.
- 3) The two sign bits are added or subtracted along with the other bits in the complement and adder circuit.



→ comparing this algorithm with signed-magnitude algorithms, it is much easy and simple to add and subtract the numbers, if '-ve' numbers are represented in signed-2's complement, so that most of the computers use signed 2's complement.

Multiplication Algorithm:

Multiplication of signed-magnitude data: The multiplication of two fixed point binary numbers in signed-magnitude representation is done by a process of successive shift and add operation.

→ The sign bit of the fixed product is determined from the sign of the multiplicand and multiplier. If they are same the sign of the final product is positive, else Negative.

ex: 11 1011

10113 1101

1011

0000

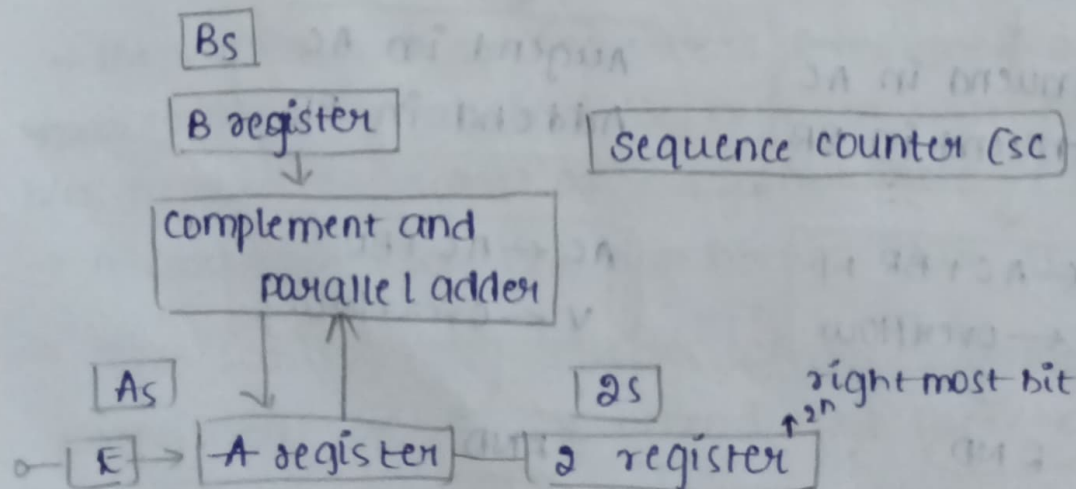
1011

1011

10001111 ← final product.

partial products

Hardware configuration for signed-magnitude Multiplication



→ The multiplier is stored in the queue register and sign of multiplier is stored in the Queue sign Q_s .

→ The sequence counter SC is initially set to a number i.e. no. of bits in the multiplier.

→ The counter is decremented by 1 after forming the each partial product.

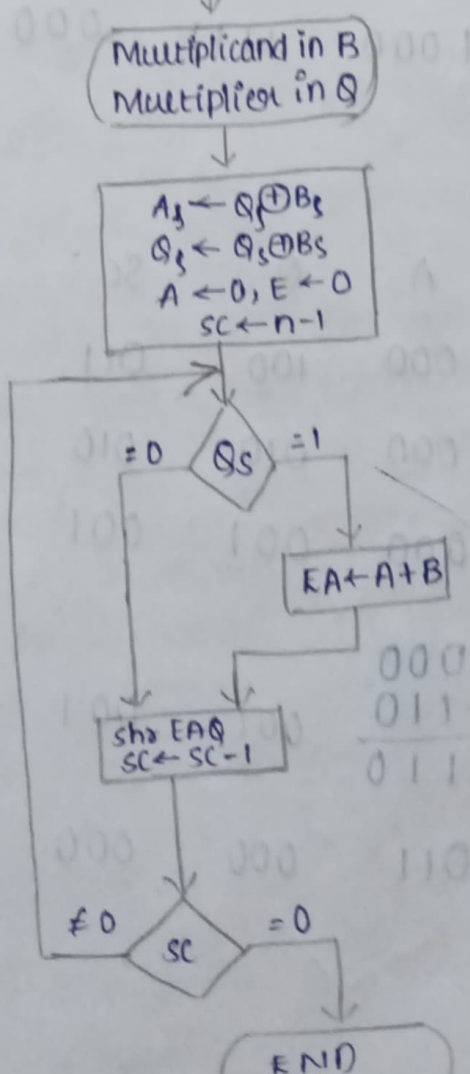
→ When the content of the counter reaches to zero the final product is formed and stops the process.

→ Initially the multiplicand in the B register (B and B_s) and multiplier in the Queue register (Q and Q_s)

→ The sum of A and B forms the partial product which stored or transfers to EA register.

→ Both partial product and multiplier are shifted to right which is represented by $SHR\ ENG$

Multiply operations



* Algorithm for flowchart for multiplication of sign magnitude numbers

$$\begin{array}{r} 000 \\ 011 \\ \hline 011 \end{array}$$

Ex: 11x13

	B	E	A	Q	SC
<u>initially</u>	1011	0	0000	1101	100
$Q_n = 1$ -Add		0	$\begin{array}{r} 1011 \\ 1101 \\ \hline 1011 \end{array}$	1101	
		0	0101	1110	011
$Q_n = 0$ SHR		0	0010	1111	010
$Q_n = 1$ Add		0	$\begin{array}{r} 0010 \\ 1011 \\ \hline 1101 \end{array}$	1111	010
SHR		0	0110	1111	001
		1	$\begin{array}{r} 1011 \\ 0001 \\ \hline 1000 \end{array}$	1111	001
$Q_n = 1$ Add SHR		0	1000	1111	000

Ex (6)x4

	B	E	A	Q	SC
	110	0	000	100	011
$Q_n = 0$ SHR		0	000	010	010
$Q_n = 0$ SHR		0	000	001	001
$Q_n = 1$ Add			$\begin{array}{r} 000 \\ 110 \\ \hline 110 \end{array}$	001	001
SHR		0	011	000	000

Booth Multiplication Algorithm:

→ The Booth Multiplication Algorithm gives processor for multiplying binary integer in sign a's complement representation.

→ This Algorithm suggest that we can reduce the no of operations required for multiplication by representing multiplier as a difference b/w two numbers.

→ It is based on the string of 0's and 1's in the multiplier.

NOTE: String of 1's in a multiplier from 2^k to 2^m treated as ~~2^{k+1}~~ $2^{k+1} - 2^m$.

ex: 14

① Binary: 001110
 2^3 to 2^1

$$\Rightarrow 2^3 \text{ to } 2^1$$

$$\Rightarrow 2^4 - 2^1$$

$$= 14$$

② $\begin{matrix} 64 & 32 & 16 & 8 & 4 & 2 & 0 \\ 62 & 1 & 1 & 1 & 1 & 1 & 0 \\ 00111110 \end{matrix}$

$$2^5 \text{ to } 2^1$$

$$\Rightarrow 2^6 - 2^1$$

$$= 62$$

This algorithm works for +ve multiplier as well as -ve multiplier in a's complement representation.

③ -14

$$+14 \rightarrow 001110$$

$$(2^6 - 2^4) + 2^2 - 2^1$$

$$15 \rightarrow 110001$$

$$2^5 \rightarrow \begin{array}{r} 11 \\ 110010 \end{array}$$

④ 58

$$00111010$$

$$2^6 - 2^3 + 2^2 - 2^1$$

$$= 58$$

⑤ -9

$$10111$$

$$(2^5 - 2^4) + 2^3 - 2^0 \Rightarrow -9$$

This is

*) Based on this the conditions for Booth multiplication:

if it is

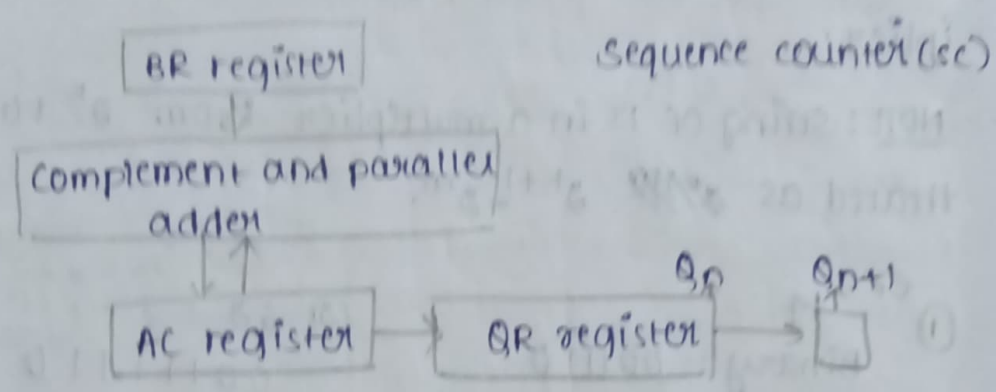
10 - sub & ASHR

01 - Add & ASHR

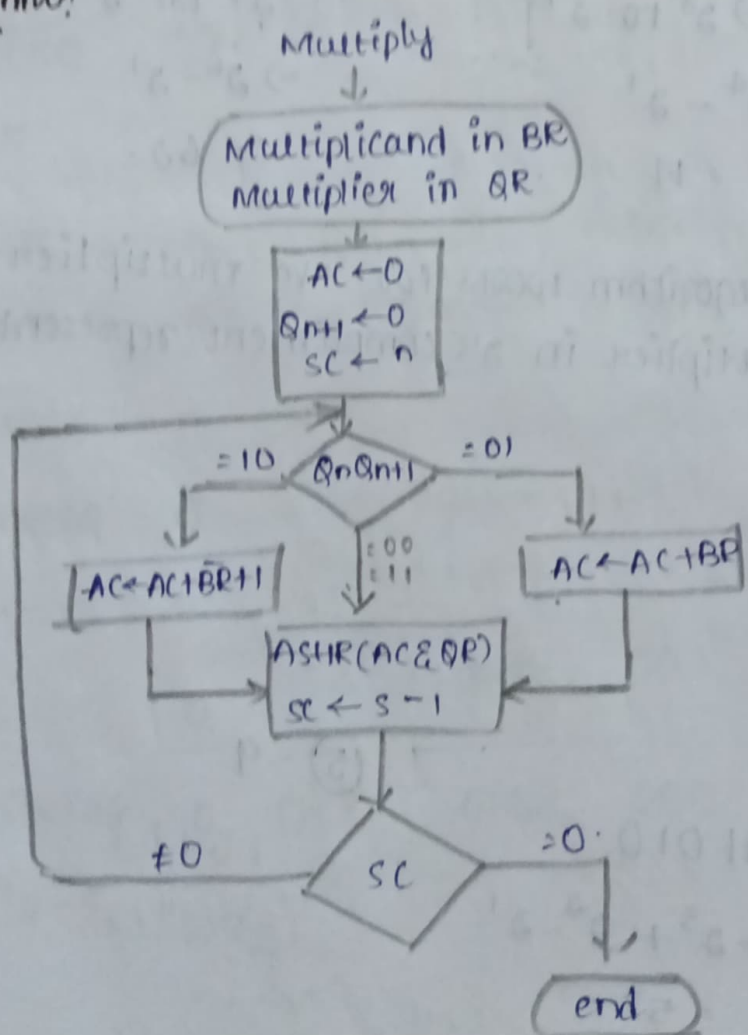
11 2 NO operation
00 } only ASHR

Hardware configuration for Booth multiplication

Algorithm:



Algorithm:



EX: 5x4
 ↓ ↓
 BR QR

BR → 0101

QR → 0100

BR + 1 → 1011

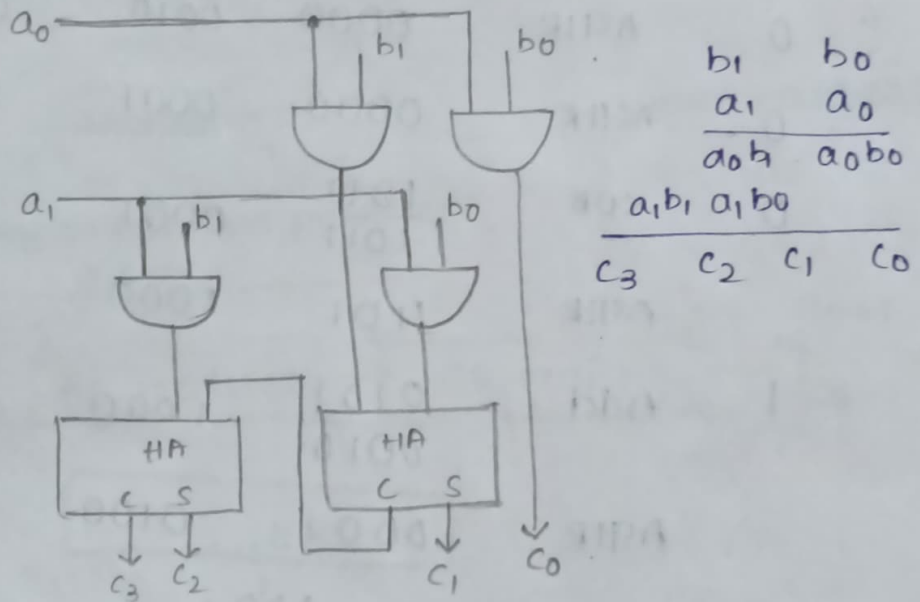
Q _n	Q _{n+1}	operation	AC	QR	Q _{n+1}	SC
0	0	initial	0000	0100	0	100
0	0	ASHR	0000	0010	0	011
0	0	ASHR	0000	0001	0	010
1	0	SUB	$\frac{1011}{1011}$	0001	0	010
		ASHR	1101	1000	1	001
0	1	Add	$\frac{0101}{1010}$ 0010	1000	1	001
		ASHR	0001	0100	0	000

+20

Array multiplier : Fast Approach:

The multiplication of 2 binary numbers can be done with one micro operation by means of a combinational circuit that generates the all partial products at a time.

ex:



→ It is the fast way of multiplying 2 numbers since all it takes the time for the signals to propagate through the logic gates that forms the multiplication array.

→ However, as the no. of bits increases, array multiplier requires large no. of logic gates and circuit complexity increases, for this reason it is not economical for higher order bits.

Division algorithm

$A/B = q$, A : Dividend, B : Divisor, q : quotient,

r : remainder

→ The division of 2 fixed point binary numbers in signed magnitude representation is done by a process of successive compare shift and subtract operations.

→ The sign of the quotient is determined from the sign of the dividend and divisor, if the signs are alike, the sign of the quotient is positive, if they are unlike, the sign of the quotient is negative.

→ The sign of the remainder is same as the sign of the dividend.

10001) 01110 000 00 (11010

1000	↓	↓	↓	↓
101	10			
100	01			
01	01	000		
	1	000	1	
		0	0110	

→ DVF condition occurs, if the higher order bits of the dividend constitute a number $>$, $>$ the divisor

ex: From above ex, if $A > B$ the DVF occurs

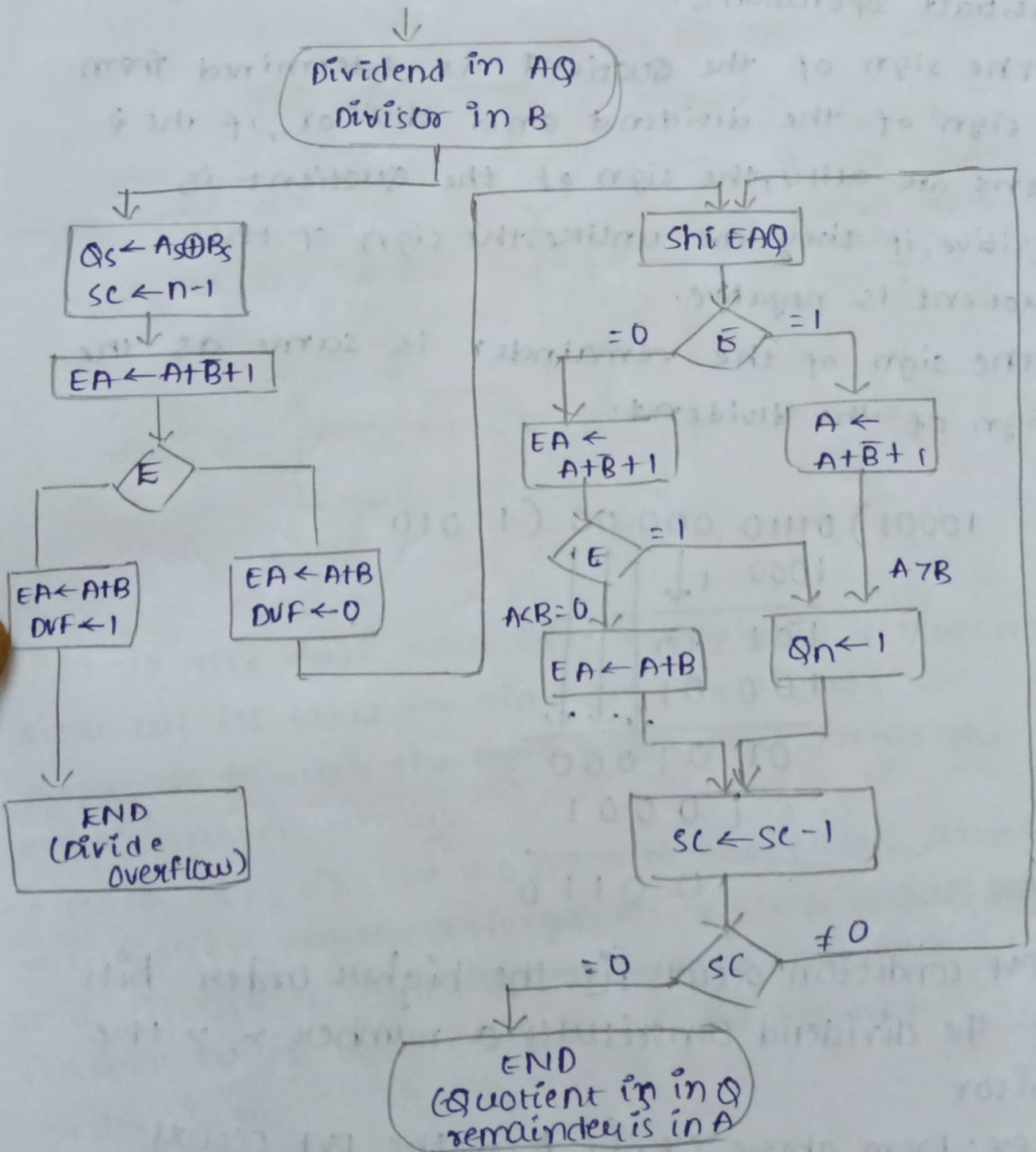
⇒ Division by "0" is also avoided.

Hardware for Division Algorithm

→ The hardware for implementing the division of sign magnitude data is same that is required for the multiplication of sign magnitude numbers.

→ The hardware configuration is same as

Divide operation.



- The information for relative magnitude is given to E
- The sequence counter is used to store the no. of bits of the divisor.
- Initially the dividend is stored in AQ and divisor is stored in Q Register.
- The sign of the result is transferred into $q_0(Q_s)$
- The count is stored in the no. of bits of the divisor. Hence, algorithm is represented as $n-1$
- The condition for DVF is checked by subtracting the divisor from the higher order bits of the dividend i.e. we have to perform $A-B$ and implemented by $A+2$'s complement of B .
- If $A < B$, no overflow condition, we continue for the magnitudes.
- The value of the dividend is restored by adding 'B' to 'A'. That is why this method is also called as restoring method.

NOTE:

There are another 2 methods for division operation

(1) Comparison Method.

(2) Non-restoring Method.